

Лабораторна робота 5

Docker

Метою лабораторної роботи є набуття практичних навичок контейнеризації Node.js-застосунків з використанням Docker шляхом розгортання REST API з лабораторної роботи №4 у контейнерному середовищі, зокрема:

- встановлення Docker та ознайомлення з основними концепціями контейнеризації;
- створення Dockerfile з multi-stage збіркою для TypeScript Express-застосунку;
- налаштування .dockerignore для оптимізації контексту збірки;
- оркестрація Express + MongoDB через Docker Compose з volumes та мережами;
- реалізація healthcheck-ендпоінта з перевіркою стану підключення до бази даних;
- налаштування healthcheck у Docker Compose для моніторингу стану сервісів.

Хід роботи

Завдання 1. Встановлення Docker

Було встановлено Docker Desktop, перевірено його встановлення та запущено тестовий контейнер для перевірки.

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.12.000 – Лр.5						
Змн.	Арк.	№ докум.	Підпис	Дата							
Розроб.		Сергійко В.М.			Звіт з лабораторної роботи №5			Літ.	Арк.	Аркуші	
Перевір.		Оринчак А. І.								1	9
Реценз.								ФІКТ, гр. ІПЗк-24			
Н. Контр.											
Зав.каф.											

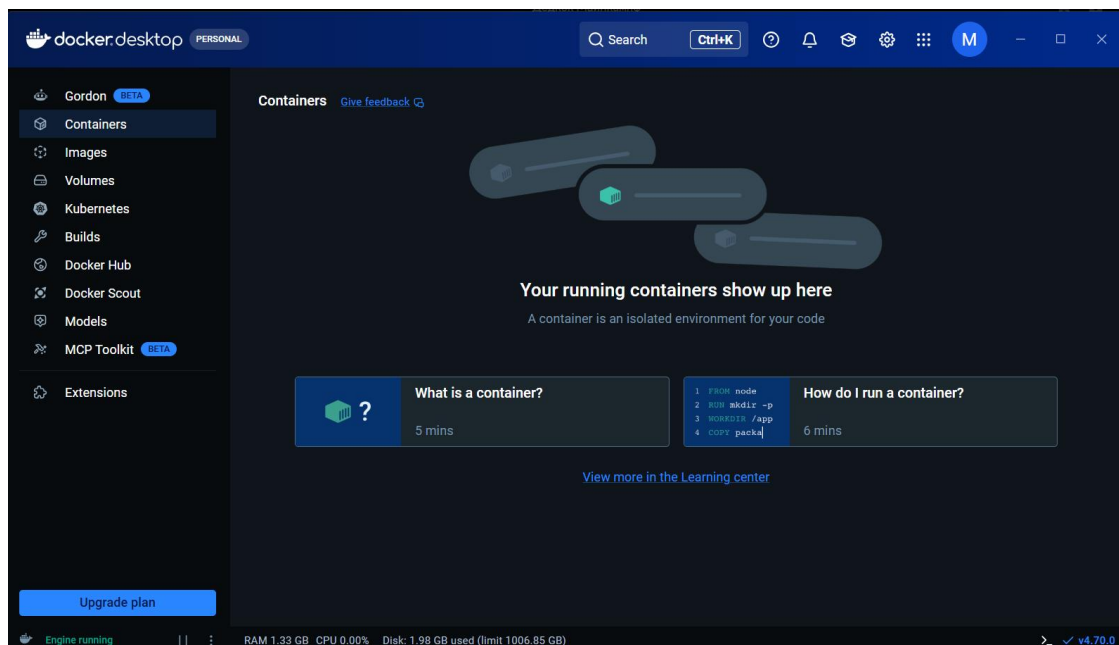


Рис.5.1 – Встановлений Docker Desktop

```
C:\WINDOWS\system32\cmd. X + v

Microsoft Windows [Version 10.0.26200.8246]
(с) Корпорація Майкрософт. Усі права захищені.

C:\Users\sergi>docker --version
Docker version 29.4.0, build 9d7ad9f

C:\Users\sergi>docker compose version
Docker Compose version v5.1.2

C:\Users\sergi>
```

Рис.5.2 – Перевірка встановлення

```
C:\WINDOWS\system32\cmd. X + v

C:\Users\sergi>docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

Рис.5.3 – Запущений тестовий контейнер

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.12.000 – Лр.5	Арк.
						2
Змн.	Арк.	№ докум.	Підпис	Дата		

Завдання 2. Dockerfile, .dockerignore та локальний запуск

Було створено файл .dockerignore та виключено файли, що не потрібні всередині контейнера.

Лістинг .dockerignore

```
.dockerignore U X
.dockerignore
1  node_modules
2  dist
3  .env
4  .git
5  .gitignore
6  Dockerfile
7  .dockerignore
8  npm-debug.log
```

Далі було створено multi-stage Dockerfile з усіма вказаними параметрами.

Лістинг Dockerfile:

```
FROM node:24 AS builder

WORKDIR /app

COPY package*.json ./

RUN npm install --legacy-peer-deps

COPY . .

RUN npm run build

FROM node:24

WORKDIR /app

COPY package*.json ./

RUN npm install --omit=dev

COPY --from=builder /app/dist ./dist

EXPOSE 3000

USER node
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.12.000 – Лр.5	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		3

```
CMD ["node", "dist/app.js"]
```

Використовуючи команду «`docker build -t myapp .`» збираємо проект. Та запускаємо його з передачею змінних з `env` файлу.

```
View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/lk9dggprmo2la19qjdxhxcz4a
PS D:\Study\Politech\25-26\2_semestr\NodeJs\Lab3-4\dotaHeroes> docker run -p 3000:3000 --env-file .env myapp
◇ injected env (0) from .env // tip: % multiple files { path: ['.env.local', '.env'] }
Успішно підключено до MongoDB
Server is running on http://localhost:3000
```

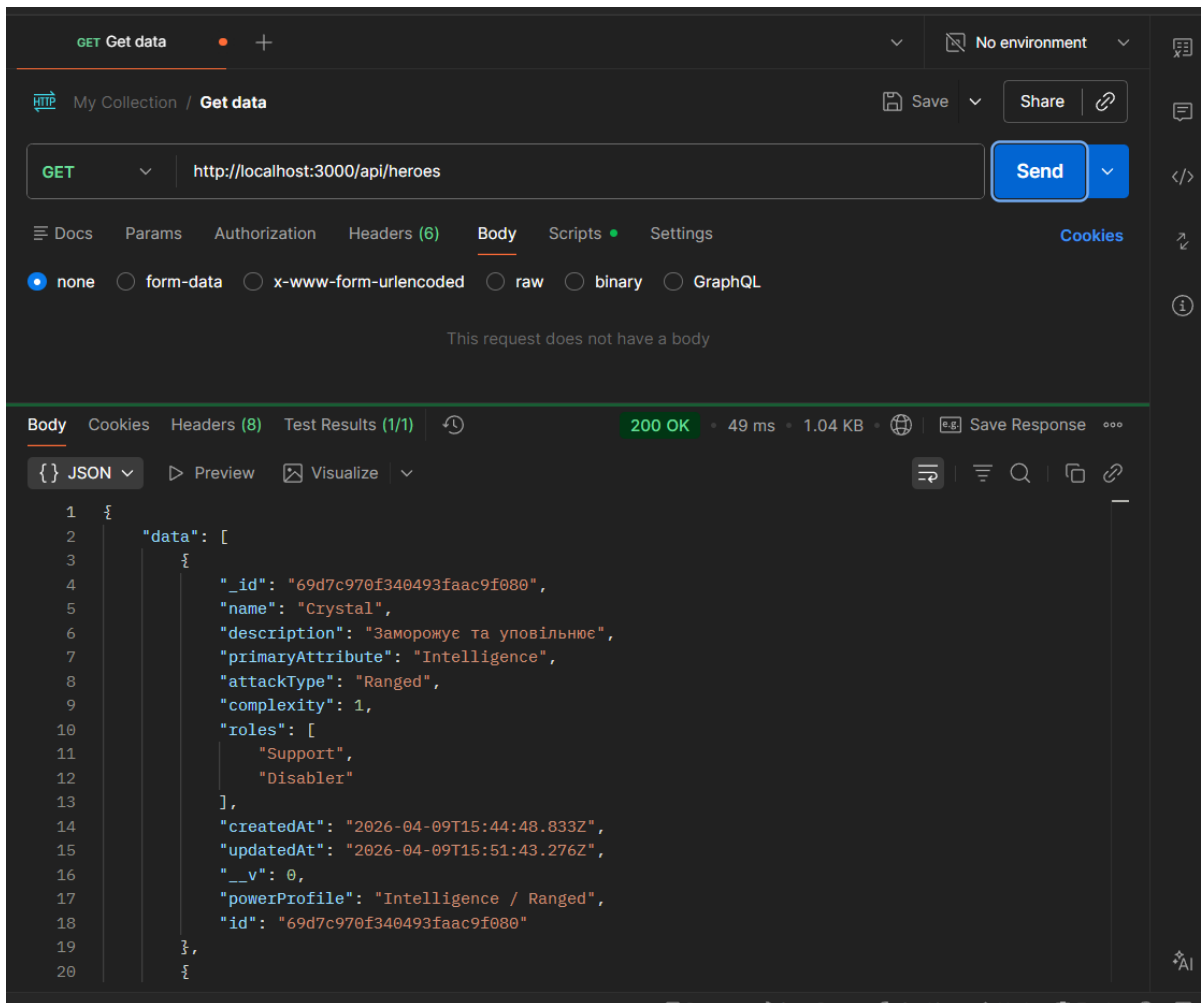


Рис.5.4 – Запуск контейнера і перевірка його роботи

Також було ознайомлено з командами управління контейнерами.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\Study\Politech\25-26\2_semestr\NodeJs\Lab3-4\dotaHeroes> docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
99277ad9d669   myapp     "docker-entrypoint.s..." 7 minutes ago  Up 7 minutes  0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp  eloquent_kalam
```

Рис.5.5 – `docker ps` – переглянути запущені контейнери.

```

PS D:\Study\Politech\25-26\2_semestr\NodeJs\Lab3-4\dotaHeroes> docker logs 99277ad9d669
◇ injected env (0) from .env // tip: multiple files { path: ['.env.local', '.env'] }
Успішно підключено до MongoDB
Server is running on http://localhost:3000
PS D:\Study\Politech\25-26\2_semestr\NodeJs\Lab3-4\dotaHeroes>

```

Рис.5.6 – docker logs – переглянути логи.

```

PS D:\Study\Politech\25-26\2_semestr\NodeJs\Lab3-4\dotaHeroes> docker stop 99277ad9d669
99277ad9d669
PS D:\Study\Politech\25-26\2_semestr\NodeJs\Lab3-4\dotaHeroes>

```

Рис.5.7 – docker stop – зупинити контейнер.

```

PS D:\Study\Politech\25-26\2_semestr\NodeJs\Lab3-4\dotaHeroes> docker rm 99277ad9d669
99277ad9d669
PS D:\Study\Politech\25-26\2_semestr\NodeJs\Lab3-4\dotaHeroes>

```

Рис.5.8 – docker rm – видалити зупинений контейнер

Завдання 3. Docker Compose

Створено compose.yaml для оркестрації Express-застосунку разом із локальним MongoDB.

Лістинг compose.yaml:

```

services:
  mongo:
    image: mongo:8
    environment:
      MONGO_INITDB_ROOT_USERNAME: admin
      MONGO_INITDB_ROOT_PASSWORD: secretpassword
      MONGO_INITDB_DATABASE: lab_database
    volumes:
      - mongo-data:/data/db

  app:
    build: .
    ports:
      - "3000:3000"
    depends_on:
      - mongo
    environment:
      NODE_ENV: development
      PORT: 3000
      MONGODB_URI:
mongodb://admin:secretpassword@mongo:27017/lab_database?authSource=admin

volumes:
  mongo-data:

```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.12.000 – Лр.5	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		5

```
PS D:\Study\Politech\25-26\2_semestr\NodeJs\Lab3-4\dotaHeroes> docker compose up --build
>>
.15.0, LE", "os": {"name": "linux", "architecture": "x64", "version": "6.6.87.2-microsoft-standard-WSL2", "type": "Linux"}, "env": {"container": {"runtime": "docker"}}
}}}}
mongo-1 | {"t":{"$date":"2026-04-23T11:30:50.199+00:00"},"s":"I", "c":"ACCESS", "id":10483900,"ctx":"conn3","msg":"Connection not authenticating","at
tr":{"client":"172.18.0.3:49240","doc":{"driver":{"name":"nodejs|Mongoose","version":"7.1.1|9.4.1"},"platform":"Node.js v24.15.0, LE","os":{"name":"linux
","architecture":"x64","version":"6.6.87.2-microsoft-standard-WSL2","type":"Linux"},"env":{"container":{"runtime":"docker"}}}}}
mongo-1 | {"t":{"$date":"2026-04-23T11:30:55.696+00:00"},"s":"I", "c":"NETWORK", "id":22943, "ctx":"listener","msg":"Connection accepted","attr":{"r
emote":"172.18.0.3:49250","isLoadBalanced":false,"uid":{"uid":{"$uid":"438a6df8-d44f-43f6-b0fd-b63ad060f8d3"},"connectionId":4,"connectionCount":4}}
mongo-1 | {"t":{"$date":"2026-04-23T11:30:55.697+00:00"},"s":"I", "c":"NETWORK", "id":51800, "ctx":"conn4","msg":"client metadata","attr":{"remote":
"172.18.0.3:49250","client":"conn4","negotiatedCompressors":[]},"doc":{"driver":{"name":"nodejs|Mongoose","version":"7.1.1|9.4.1"},"platform":"Node.js v24
.15.0, LE","os":{"name":"linux","architecture":"x64","version":"6.6.87.2-microsoft-standard-WSL2","type":"Linux"},"env":{"container":{"runtime":"docker"}}
}}}}
mongo-1 | {"t":{"$date":"2026-04-23T11:30:55.697+00:00"},"s":"I", "c":"ACCESS", "id":6788604, "ctx":"conn4","msg":"Auth metrics report","attr":{"metr
ic":"acquireUser","micros":0}}
mongo-1 | {"t":{"$date":"2026-04-23T11:30:55.700+00:00"},"s":"I", "c":"ACCESS", "id":5286306, "ctx":"conn4","msg":"Successfully authenticated","attr
":{"client":"172.18.0.3:49250","isSpeculative":true,"isClusterMember":false,"mechanism":"SCRAM-SHA-256","user":"admin","db":"admin","result":0,"metrics":{"
conversation_duration":{"micros":2437,"summary":{"0":{"step":1,"step total":2,"duration micros":169},"1":{"step":2,"step total":2,"duration micros":29}}
},"doc":{"driver":{"name":"nodejs|Mongoose","version":"7.1.1|9.4.1"},"platform":"Node.js v24.15.0, LE","os":{"name":"linux","architecture":"x64","versio
n":"6.6.87.2-microsoft-standard-WSL2","type":"Linux"},"env":{"container":{"runtime":"docker"}}},"extraInfo":{}}}
mongo-1 | {"t":{"$date":"2026-04-23T11:30:55.702+00:00"},"s":"I", "c":"NETWORK", "id":6788700, "ctx":"conn4","msg":"Received first command on ingress
connection since session start or auth handshake","attr":{"elapsedMillis":2}}
mongo-1 | {"t":{"$date":"2026-04-23T11:31:39.677+00:00"},"s":"I", "c":"WITCHPT", "id":22430, "ctx":"Checkpoint", "msg":"WiredTiger message","attr":
{"message":{"ts_sec":1776943899,"ts_usec":677813,"thread":"1:0x7a7563d016c0","session_name":"WT_SESSION.checkpoint","category":"WT_VERB_CHECKPOINT_PROGRE
SS","log_id":1000000,"category_id":7,"verbose_level":"INFO","verbose_level_id":0,"msg":"saving checkpoint snapshot min: 10, snapshot max: 10 snapshot cou
nt: 0, oldest timestamp: (0, 0), meta checkpoint timestamp: (0, 0) base write gen: 7"}}}
|
View in Docker Desktop View Config Enable Watch Detach
```

Рис.5.9 – Запуск командою docker compose up --build

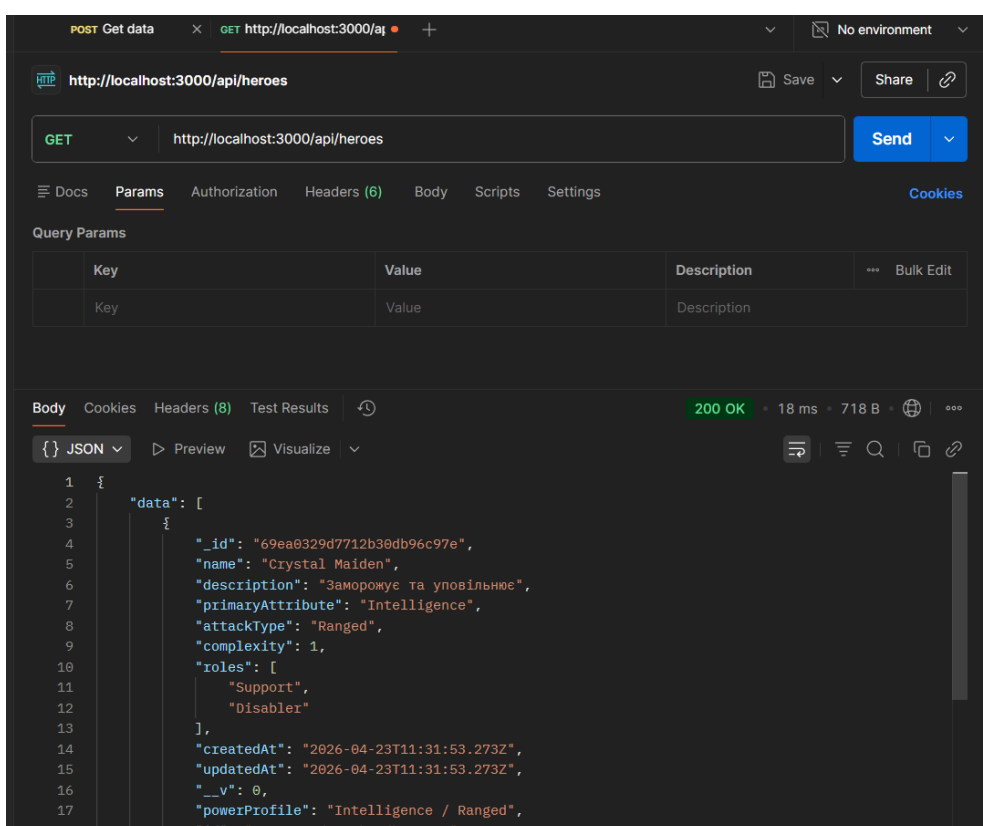


Рис.5.10 – Перевірка роботи

Сервіси працюють коректно, записи у БД додаються та повертаються, тепер необхідно перевірити цілісність даних після відключення та повторного увімкнення сервісів.

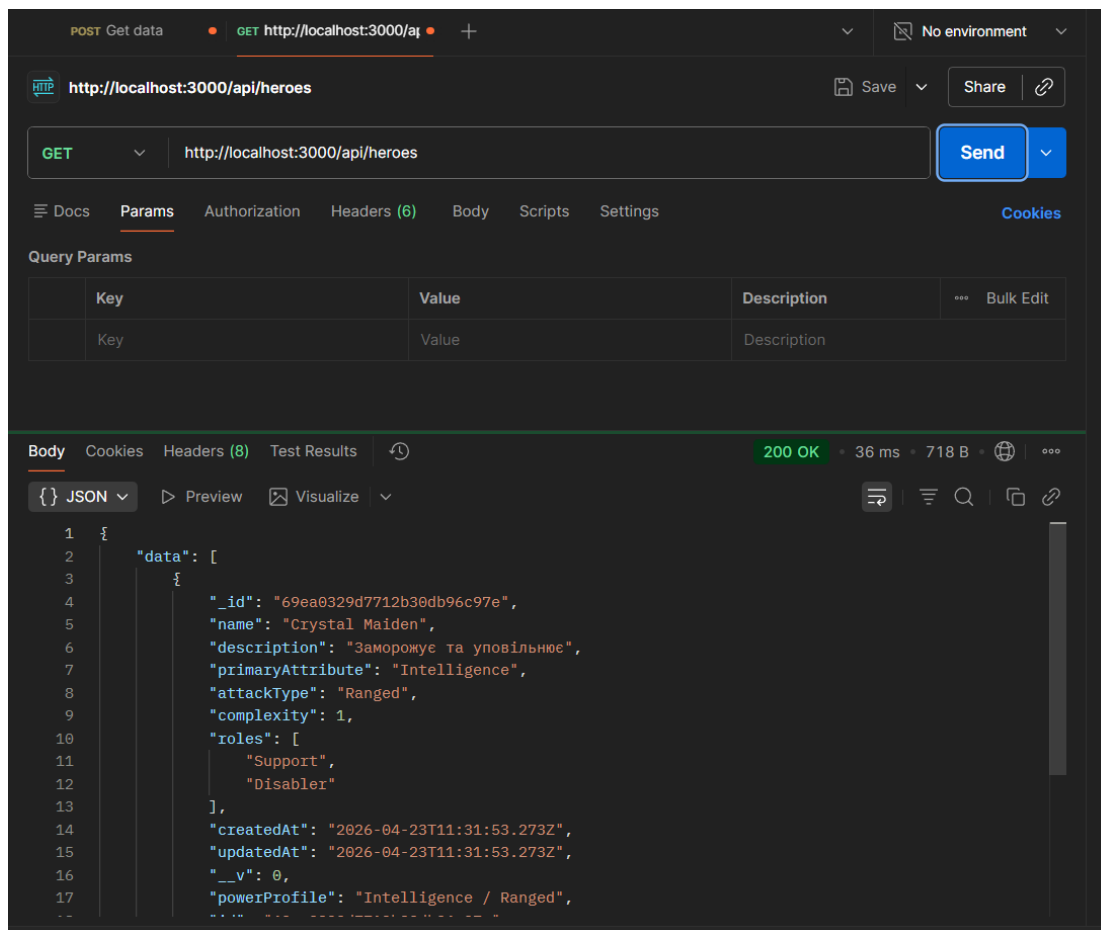


Рис.5.11 – Коректна робота сервісів після перезапуску

Також було ознайомлено з командами Docker Compose:

```
PS D:\Study\Politech\25-26\2_semestr\NodeJs\Lab3-4\dotaHeroes> docker compose ps
NAME                IMAGE              COMMAND                  SERVICE    CREATED          STATUS          PORTS
dotaheroes-app-1    dotaheroes-app    "docker-entrypoint.s..." app         About a minute ago Up 9 seconds    0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
dotaheroes-mongo-1 mongo:8            "docker-entrypoint.s..." mongo       About a minute ago Up 9 seconds    27017/tcp
```

Рис.5.12 – docker compose ps – статус сервісів

```
PS D:\Study\Politech\25-26\2_semestr\NodeJs\Lab3-4\dotaHeroes> docker compose logs -f app
app-1 | injected env (0) from .env // tip: % override existing { override: true }
app-1 | Успішно підключено до MongoDB
app-1 | Server is running on http://localhost:3000
app-1 |
app-1 | Closing server after ${signal}
app-1 | HTTP сервер закрито
app-1 | MongoDB відключено
app-1 | З'єднання з MongoDB закрито
app-1 | injected env (0) from .env // tip: % multiple files { path: ['.env.local', '.env'] }
app-1 | Успішно підключено до MongoDB
app-1 | Server is running on http://localhost:3000
```

Рис.5.13 – docker compose logs -f app – логи сервісу app у реальному часі

```
PS D:\Study\Politech\25-26\2_semestr\NodeJs\Lab3-4\dotaHeroes> docker compose down
[+] down 3/3
✓ Container dotaheroes-app-1    Removed
✓ Container dotaheroes-mongo-1  Removed
✓ Network dotaheroes_default    Removed
PS D:\Study\Politech\25-26\2_semestr\NodeJs\Lab3-4\dotaHeroes>
```

Рис.5.13 – docker compose down – зупинити та видалити контейнери

```

PS D:\Study\Politech\25-26\2_sestr\NodeJs\Lab3-4\dotaHeroes> docker compose down -v
[+] down 1/1
✓ Volume dotaheroes_mongo-data Removed
PS D:\Study\Politech\25-26\2_sestr\NodeJs\Lab3-4\dotaHeroes>

```

Рис.5.14 – docker compose down -v – також видалити volumes

Завдання 4. Healthcheck-ендпоінт

Було додано маршрут GET /health до app.ts. Зареєстровано його перед усіма іншими middleware та маршрутами, щоб healthcheck працював безумовно.

Лістинг app.ts:

```

import express from "express";
import cors from "cors";
import heroRouter from "./routes/hero";
import { errorHandler } from "./middleware/errorHandler";
import mongoose from "mongoose";

const app = express();

app.get("/health", (req, res) => {
  if (mongoose.connection.readyState === 1) {
    res.status(200).json({ status: "OK", database: "connected" });
  } else {
    res
      .status(503)
      .json({ status: "Service Unavailable", database: "disconnected" });
  }
});

app.use(cors());
app.use(express.json());

app.use("/api/heroes", heroRouter);

app.use(errorHandler);

export default app;

```

Додано healthcheck до сервісу app у compose.yaml – Docker повинен періодично перевіряти ендпоінт /health та відслідковувати стан контейнера. Додано healthcheck до сервісу mongo – перевірка доступності MongoDB через вбудовану команду ping. Оновлено depends_on для сервісу app, щоб він чекав на здоровий стан MongoDB перед запуском.

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.12.000 – Лр.5	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		8

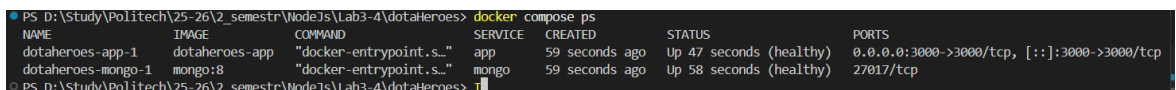
Лістинг compose.yaml:

```
services:
  mongo:
    image: mongo:8
    environment:
      MONGO_INITDB_ROOT_USERNAME: admin
      MONGO_INITDB_ROOT_PASSWORD: secretpassword
      MONGO_INITDB_DATABASE: lab_database
    volumes:
      - mongo-data:/data/db
    healthcheck:
      test: ["CMD", "mongosh", "--eval", "db.adminCommand('ping')"]
      interval: 10s
      timeout: 5s
      retries: 3

  app:
    build: .
    ports:
      - "3000:3000"
    depends_on:
      mongo:
        condition: service_healthy
    environment:
      NODE_ENV: development
      PORT: 3000
      MONGODB_URI:
        mongodb://admin:secretpassword@mongo:27017/lab_database?authSource=admin
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:3000/health"]
      interval: 15s
      timeout: 5s
      retries: 3

volumes:
  mongo-data:
```

Далі переконаємось у наявності відповідного статусу в сервісів.



```
PS D:\Study\Politech\25-26\2 semestr\NodeJs\Lab3-4\dotaHeroes> docker compose ps
NAME                IMAGE              COMMAND              SERVICE  CREATED      STATUS              PORTS
dotaheroes-app-1    dotaheroes-app     "docker-entrypoint.s..." app       59 seconds ago Up 47 seconds (healthy) 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
dotaheroes-mongo-1 mongo:8            "docker-entrypoint.s..." mongo     59 seconds ago Up 58 seconds (healthy) 27017/tcp
```

Рис.5.15 – Робота обох сервісів зі статусом healthy

Висновок: Під час виконання лабораторної роботи було набуто практичних навичок роботи з контейнеризації Node.js-застосунків з використанням Docker шляхом розгортання REST API з лабораторної роботи №4 у контейнерному середовищі.

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.12.000 – Лр.5	Арк.
						9
Змн.	Арк.	№ докум.	Підпис	Дата		